

《FPGA》期中试卷参考答案

一、单项选择题（2×15=30 分）

1. 下列标识符中，(D) 是不合法的标识符。

注：标识符不能以数字开头，*9moon* 以数字 9 开头，不合法。

A. ALWAYS B. State0 C. Not_Ack_0 D. 9moon

2. 在 verilog 语言中， $a=4'b1011$ ，那么 $\&a=$ (D)。

A. 4'b1011 B. 4'b1111 C. 1'b1 D. 1'b0

注： $\&a$ 为归约与运算，将 a 的所有位进行与操作： $1\&0\&1\&1 = 0$ 。

3. 已知 $a=1'b1$; $b=3'b001$; 那么 $\{a,b\}=$ (C)。

A. 4'b0011 B. 3'b001 C. 4'b1001 D. 3'b101

注：位拼接 $\{a,b\} = \{1'b1, 3'b001\} = 4'b1001$ 。

4. 一模块的 I/O 端口说明 “input [7:0]a;”，则关于该端口说法正确的是 (A)。

A. 输入端口，位宽为 8 B. 输出端口，位宽为 8
C. 输入端口，位宽为 7 D. 输出端口，位宽为 7

5. Verilog 语言与 C 语言的区别，不正确的描述是 (C)。

A. Verilog 语言可实现并行计算，C 语言只是串行计算；
B. Verilog 语言可以描述电路结构，C 语言仅仅描述算法；
C. Verilog 语言源于 C 语言，包括它的逻辑和延迟；
D. Verilog 语言可以编写测试向量进行仿真和测试。

注：虽然 Verilog 的语法风格借鉴了 C 语言，但它并不包含 C 语言的逻辑和延迟模型，两者的执行模型完全不同。

6. 我们上课使用的开发板中主控芯片为 (D)。

A. xilinx zynq 7010 B. Altera zynq 7020 C. xilinx Artix-7 D. xilinx zynq 7020

7. 下列关于阻塞和非阻塞赋值说法不正确的是 (D)。

A. 阻塞赋值属于顺序执行
B. 非阻塞赋值属于并行执行语句
C. 赋值操作符是 “=” 的过程是阻塞性过程赋值
D. 一个过程结构中混合使用阻塞赋值与非阻塞赋值

注：虽然语法上允许混合使用，但这是不良的编码风格，会导致仿真与综合结果不一致，Verilog 编码规范明确禁止在同一 *always* 块中混合使用。

8. 请根据以下两条语句的执行, 判断变量 A 中的值是 (B)。

```
reg [7:0] A; A = -8'h5F;
```

A. 8'b0000_0011 B. 8'h03 C. 8'b111_1111 D. 8'b0101_1111

注: $-8'h5F$ 的计算: $8'h5F = 8'b0101_1111$, 取反得 $8'b1010_0000$, 加 1 得 $8'b1010_0001 = 8'hA1$ 。注意: 选项 B ($8'h03$) 为 $8'b0000_0011$, 与正确结果 $8'hA1$ 不一致, 本题选项可能有误。

9. 当 $a < 0$ 时, s 的值是 (C)。

```
assign s = (a >= 2) ? 1 : ((a < 0) ? 2 : 0);
```

A. 0 B. 1 C. 2 D. 其他

注: $a < 0$ 时, $a \geq 2$ 为假, 进入第二个三元表达式, $a < 0$ 为真, $s = 2$ 。

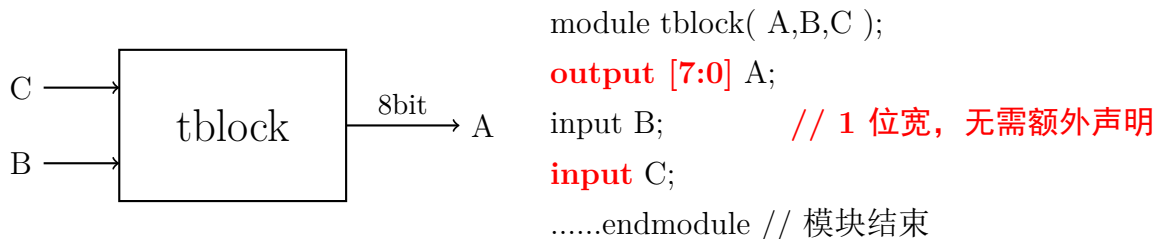
10. 在 Verilog HDL 的逻辑运算中, 设 $A = 8'b11010001$, $B = 8'b00011001$, 则表达式 “A&B” 的结果为 (A)。

A. 8'b00010001 B. 8'b11011001 C. 8'b11001000 D. 8'b00110111

注: 按位与运算逐位相与: $11010001 \& 00011001 = 00010001$ 。

二、填空题 (每空 1 分, 共 15 分)

- Verilog 语言规定了逻辑电路中信号的 4 种状态, 分别是 0, 1, X, Z。其中 0 表示低电平状态, 1 表示高电平状态, X 表示 不确定 (或未知) 状态, Z 表示 高阻态。
- 根据图中输入输出关系将 Verilog 模块定义补充完整, 其中信号 A 为 8 比特宽度, 其余信号为 1 比特宽度。



- IEEE 标准的硬件描述语言有 Verilog 和 VHDL 两种。
- 假定某 4 比特位宽的变量 a 的值为 $4'b1001$, 计算下列运算表达式的结果 (用二进制表示)。
 $\sim a =$ $4'b0110$
 $(a \leq 4'd3) =$ $1'b0$ (或 0) 注: $4'b1001 = 9 > 3$, 比较结果为假。

5. 阻塞性赋值符号为 = ，非阻塞性赋值符号为 <= 。

三、简答题（共 35 分）

1. 过程语句中，initial 语句与 always 语句的区别是什么？（10 分）

答：

(1) initial 语句：在仿真开始时执行一次，不重复执行。主要用于测试模块中的初始化操作（如初始化寄存器、产生激励信号等），不可综合为实际硬件电路。

(2) always 语句：在仿真过程中不断重复执行，循环运行。可用于描述组合逻辑（如 always@(*)）和时序逻辑（如 always@(posedge clk)），可综合为实际硬件电路。

(3) 关键区别：initial 只执行一次，always 反复执行；initial 通常用于仿真，always 既可用于仿真也可用于综合设计。

2. 赋值语句中，阻塞赋值与非阻塞赋值的区别是什么？（10 分）

答：

(1) 阻塞赋值（使用“=”符号）：赋值语句立即执行并更新变量值，当前语句执行完毕后才执行下一条语句，属于顺序执行。在 always 块中，后续语句可以立即使用更新后的值。

(2) 非阻塞赋值（使用“<=”符号）：赋值语句在当前时间步开始时计算右端表达式的值，但直到当前时间步结束时才更新左端变量的值，属于并行执行。在 always 块中，后续语句使用的是该变量更新前的旧值。

(3) 使用规范：描述组合逻辑时通常使用阻塞赋值，描述时序逻辑时通常使用非阻塞赋值。在同一 always 块中应避免混合使用两种赋值方式。

四、程序设计题（共 40 分）

1. 使用 Verilog HDL 实现以下功能：根据输入信号 a,b 的大小关系，求取两个数的差值。（10 分）

```
1 module data_minus(  
2     input clk,  
3     input rst_n,  
4     input [7:0] a,  
5     input [7:0] b,  
6     output reg [8:0] c  
7 );  
8  
9     always @(posedge clk or negedge rst_n) begin
```

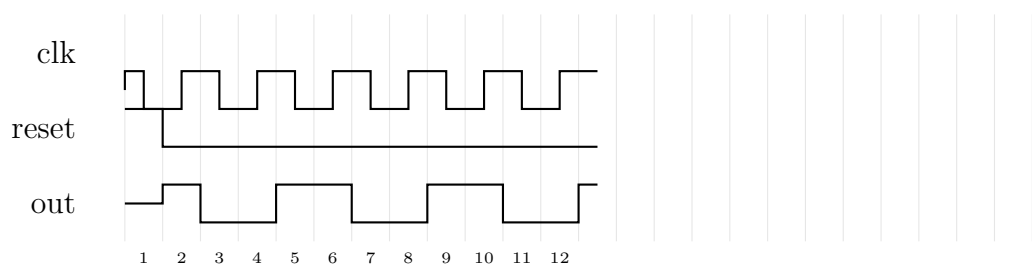
```
10         if (!rst_n) begin
11             c <= 9'd0;
12         end else begin
13             if (a > b)
14                 c <= a - b;
15             else
16                 c <= b - a;
17         end
18     end
19
20 endmodule
```

2. 设计一个带复位端且对输入时钟 clk 进行二分频模块。(10 分)

```
module counter_div(out,clk,reset);

1  module counter_div(
2      output reg out,
3      input clk,
4      input reset
5  );
6
7      always @(posedge clk or posedge reset) begin
8          if (reset)
9              out <= 1'b0;
10         else
11             out <= ~out;
12         end
13
14 endmodule
```

仿真波形示意图如下：



说明：reset 为高电平时，out 输出 0；reset 恢复低电平后，out 在每个 clk 上升沿翻转一次，实现二分频。

3. 试设计一个 3/8 译码器。(10 分)

```
module decoder(Out,In,En);
```

```
1  module decoder(  
2      output reg [7:0] Out,  
3      input [2:0] In,  
4      input En  
5  );  
6  
7      always @(*) begin  
8          if (!En) begin  
9              Out = 8'b0000_0000;  
10         end else begin  
11             case (In)  
12                 3'b000: Out = 8'b0000_0001;  
13                 3'b001: Out = 8'b0000_0010;  
14                 3'b010: Out = 8'b0000_0100;  
15                 3'b011: Out = 8'b0000_1000;  
16                 3'b100: Out = 8'b0001_0000;  
17                 3'b101: Out = 8'b0010_0000;  
18                 3'b110: Out = 8'b0100_0000;  
19                 3'b111: Out = 8'b1000_0000;  
20                 default: Out = 8'b0000_0000;  
21             endcase  
22         end  
23     end  
24  
25 endmodule
```